

CSA0613 - Design and Analysis of Algorithms

CO2 AT3 DEBUGGING & OPTIMISATION TASK

Divyesh S P (192424147)

Debugging and Optimization of Linear Search Algorithm

Aim

To identify and correct the logical error in the given Linear Search algorithm, ensure complete traversal of the array, optimize the implementation for clarity and correctness, and analyze its time complexity.

Problem Statement

The given Linear Search program terminates prematurely because of an incorrect `return` statement placed inside the loop. This causes the search to stop after checking only the first element, even if the required element exists later in the array. The task is to identify the logical error, debug the code, optimize the implementation, and analyze its time complexity.

Given Code (Incorrect)

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] != key:  
            return -1    # ERROR  
    else:  
        return i
```

Logical Error

The statement `return -1` is placed inside the `for` loop.

If the first element does not match the key, the function immediately returns `-1` without checking the remaining elements. As a result, valid elements present later in the array are never searched.

Why Valid Elements May Not Be Found?

Consider the array:

Array = [10, 20, 30, 40]

Key = 30

Execution:

- Compare 10 with 30 → Not equal.
- The function immediately returns `-1`.
- Elements 20, 30, and 40 are never checked.

Although the key exists, the program incorrectly reports that it is not found.

Corrected Code

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```

Optimized Code

```
def linear_search(arr, key):  
    for index, value in enumerate(arr):  
        if value == key:  
            return index  
    return -1  
  
arr = [10, 20, 30, 40, 50]  
key = 30  
  
result = linear_search(arr, key)  
  
if result != -1:  
    print("Element found at index:", result)  
else:  
    print("Element not found")
```

Algorithm

1. Start.
2. Input the array and the key element.
3. Traverse the array from the first element to the last.
4. Compare each element with the key.
5. If the key matches, return its index.
6. Continue searching until all elements are checked.
7. If the key is not found after complete traversal, return -1.
8. Stop.

Execution

```
[1] ✓ Os ▶ def linear_search(arr, key):
            for index, value in enumerate(arr):
                if value == key:
                    return index
            return -1

            # Input array
            arr = [10, 20, 30, 40, 50]

            # Element to search
            key = 30

            # Function call
            result = linear_search(arr, key)

            # Display result
            if result != -1:
                print("Element found at index:", result)
            else:
                print("Element not found")

            ... Element found at index: 2
```

Optimization

Before Optimization

- Search stopped after checking the first unmatched element.
- Returned -1 immediately.
- Did not traverse the complete array.
- Incorrect implementation.

After Optimization

- Searches every element until the key is found.
- Returns the index immediately when the key is found.

- Returns -1 only after checking all elements.
- Uses `enumerate()` to improve readability and simplify the code.

Time Complexity Analysis

Let **n** be the number of elements in the array.

Best Case

The key is found at the first position.

Time Complexity: $O(1)$

Average Case

The key is found somewhere in the middle of the array.

Time Complexity: $O(n)$

Worst Case

The key is at the last position or is not present in the array.

Time Complexity: $O(n)$

Space Complexity

The algorithm uses only a few extra variables regardless of the input size.

Space Complexity: $O(1)$

Advantages

- Simple and easy to implement.
- Works with both sorted and unsorted arrays.
- Requires no extra memory.
- Easy to understand and debug.

Disadvantages

- Inefficient for large datasets.
- Searches elements sequentially.
- Slower than Binary Search on sorted arrays.

Applications

- Searching elements in small datasets.
- Student record search.
- Employee record management.
- Inventory management systems.
- Searching names, IDs, or values in unsorted lists.

Result

The logical error causing premature termination in the Linear Search algorithm was successfully identified and corrected. The optimized implementation performs a complete traversal of the array before concluding that an element is absent. The corrected algorithm produces accurate results with a time complexity of **$O(1)$** in the best case and **$O(n)$** in the average and worst cases, while maintaining a constant space complexity of **$O(1)$** .